

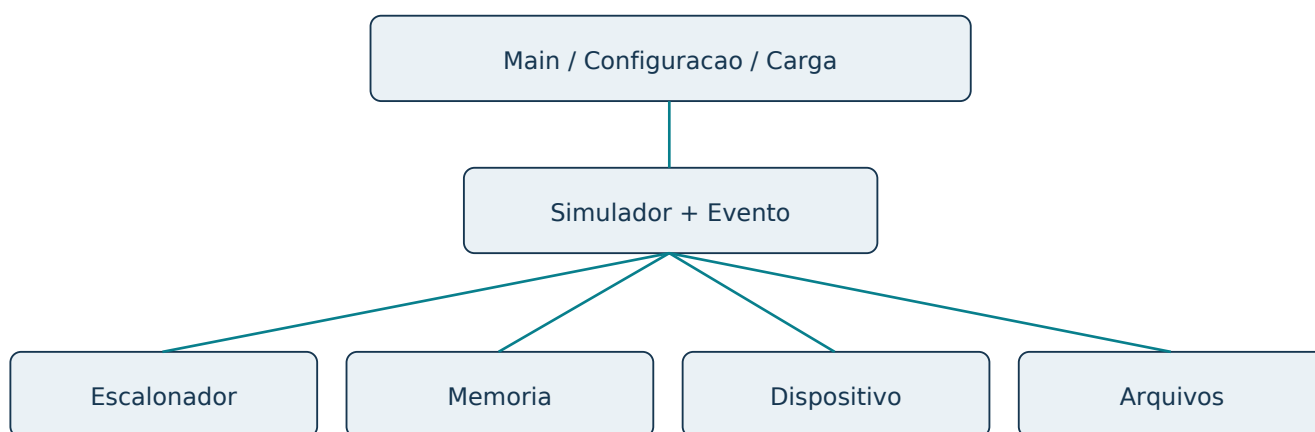
SIMULADOR DIDÁTICO DE SISTEMAS OPERACIONAIS

Relatório técnico • implementação CLI com alvo Java 27
16 de setembro de 2026

01 / Objetivo e escopo

O projeto reproduz uma CPU, processos com múltiplas threads, paginação, dispositivos e um sistema de arquivos hierárquico. O tempo é lógico: a velocidade do computador hospedeiro não interfere nas métricas. A carga é externa ao código e os eventos permitem reconstruir a execução.

Foram implementados os mecanismos obrigatórios do enunciado Trabalho SO.pdf. A prioridade é não preemptiva; o menor número representa maior prioridade. A interface gráfica foi acrescentada como extensão de apresentação; seu escopo está no adendo final. O projeto usa somente a biblioteca padrão de Java e não cria threads reais para simular concorrência.



Processo (PCB), ThreadSimulada (TCB) e Metricas integram os mecanismos.

O núcleo coordena os mecanismos; cada mecanismo concentra sua própria regra. Processo e ThreadSimulada exercem os papéis de PCB e TCB. O diagrama completo de classes está em docs/diagrama-classes.mmd e as convenções detalhadas em docs/arquitetura.md.

Clareza do código. Nomes de domínio em português, loops explícitos, ausência de camadas artificiais e dependências dispensáveis seguem o Markdown fornecido. As entidades são internas ao pacote; estados e filas têm acesso controlado. Não foi usada herança sem relação de domínio.

Autoria e apoio. Esta implementação e este relatório foram produzidos com assistência do ChatGPT/Codex, declarada no README. O grupo deve revisar e compreender integralmente o material. Não foram criados nomes de autores, participação fictícia ou histórico artificial de commits.

02 / Clock, estados e escalonamento

O próximo evento determina o avanço do clock. No mesmo instante, a ordem é: conclusões de E/S e paginação; chegadas; término de unidade da CPU; fim da troca de contexto. Dentro do mesmo tipo vale a sequência de inserção. Na ausência de trabalho, o clock salta diretamente ao próximo evento.

Unidade	Informações e regra
PCB / Processo	ID, prioridade, chegada, estado agregado, contexto da última thread, espaço lógico, threads e descritores locais.
TCB / ThreadSimulada	ID, processo dono, estado, PC, restante do surto, acumulador, pilha, caixa de entrada e tempos.
Estados de thread	NOVO → PRONTO → EXECUTANDO; da execução pode ir a PRONTO, BLOQUEADO ou FINALIZADO; BLOQUEADO retorna a PRONTO.
Estado de processo	Derivado das threads. Finaliza somente quando todas terminam e não resta E/S pendente. Cada alteração registra causa.

FCFS escolhe a primeira entrada na fila. RR usa a mesma ordem e devolve a thread ao final da fila ao esgotar o quantum. Prioridade acrescenta o valor numérico antes da ordem de entrada. Empates seguem entrada e, por último, nome completo. Uma chegada com prioridade maior não interrompe a thread atual.

Cada despacho, inclusive o primeiro ou para a mesma thread, cobra o custo de contexto. A escolha ocorre ao fim desse intervalo e considera chegadas ocorridas durante a troca. Não se executa carga útil na troca. Bloqueio e término têm precedência sobre preempção quando coincidem com o limite do quantum.

CPU N executa N unidades; cada outra instrução custa uma unidade para emitir a operação. Faltas e E/S têm tempo de serviço adicional. Se a última instrução bloquear, a thread retorna à fila e encerra no próximo despacho, sem nova unidade útil. Essa última troca também é contabilizada.

03 / Definições das métricas

Retorno = finalização – chegada. Resposta = início da primeira instrução – chegada. Espera soma intervalos PRONTO, incluindo a espera por contexto. Por processo, a resposta é a primeira das threads e a espera informada é a soma das esperas das threads, não o tempo de permanência do PCB em PRONTO.

Utilização da CPU = unidades úteis / clock final. Sobrecarga é apresentada separadamente; ociosidade = total – útil – sobrecarga. Utilização de cada dispositivo = serviço ocupado / clock final. Throughput = processos finalizados / clock final. Taxa de faltas = faltas / referências. O horizonte começa em zero e inclui a drenagem das requisições assíncronas.

04 / Memória, dispositivos e arquivos

Paginação. O endereço lógico é dividido em página e deslocamento. Cada processo tem tabela própria, que guarda presença e moldura. A tradução usa moldura $\times$ tamanho da página + deslocamento. Acertos não alteram a ordem FIFO global. Na falta, a thread bloqueia e entra na fila serial do paginador.

Ao concluir o serviço, usa-se moldura livre ou a vítima mais antiga. A tabela da vítima é invalidada e a nova página é instalada. A referência é concluída pelo evento do paginador, sem recontagem ou repetição ao retomar a thread. Pedidos simultâneos à mesma página mantêm seus atendimentos, mas reutilizam a página se ela já estiver residente. Finalizar um processo libera suas molduras.

E/S. Disco (bloco) e terminal (caractere) têm filas FCFS e tempos independentes. B bloqueia; NB permite prosseguir. A conclusão produz interrupção e entrega dispositivo:id:OK à caixa de entrada do TCB. Não há polling. Se todas as threads já terminaram, o processo aguarda os pedidos assíncronos antes de liberar recursos.

Arquivos. A raiz contém nós de diretório e arquivo, com nome, tipo, permissões, tamanho e instantes lógicos. Há criação, abertura, leitura, escrita, fechamento, remoção e listagem. Cada abertura associa descritor local a entrada global com nó, modo e cursor. Threads compartilham os descritores do processo; processos diferentes têm espaços locais independentes.

O volume usa 128 blocos encadeados de 16 unidades UTF-16. Cada bloco aponta para o próximo; o nó aponta para o primeiro. A estratégia evita exigir blocos contíguos, mas impõe percorrer a cadeia para acessar dados. A escrita verifica capacidade antes da alteração, mantendo o conteúdo anterior se faltar espaço. O tamanho é lógico, não bytes UTF-8.

Permissões r/w são verificadas na abertura e nas operações. Remoção de arquivo aberto e de diretório não vazio é rejeitada. Fechar ou encerrar o processo remove a associação local/global. As operações de arquivos são síncronas de uma unidade, sem gerar pedidos no dispositivo disco; o dispositivo é exercitado explicitamente pelas instruções IO. O volume não persiste entre execuções.

05 / Desenho dos experimentos

Os sete cenários são executados por scripts/experimentos.py. Mantêm-se troca = 1, página = 16, falta = 5, disco = 4, terminal = 2 e semente = 42. As comparações de política e quantum usam a mesma carga mista, com quatro molduras. A comparação de memória usa uma carga específica e FCFS. Não há geração aleatória; a semente é registrada para explicitar a configuração.

Os números seguintes foram obtidos na execução de referência em JDK 17, anterior à migração do alvo para Java 27. A lógica foi preservada, mas esses números ainda não foram reproduzidos em JDK 27. Tempos estão em unidades lógicas; médias de espera/resposta são por thread, retorno médio é por processo. As utilizações aparecem como porcentagem apenas neste relatório; o CSV usa proporções.

06 / Experimento 1: políticas de CPU

Carga mista idêntica, quatro molduras, quantum 3 (aplicável somente ao RR). O programa contém quatro threads em três processos; executa 54 unidades úteis e seis referências à memória em todas as políticas.

Política	Total	Trocas	Resposta média	Espera média	Retorno médio
FCFS	70	13	17,75	30,25	63,33
RR, q=3	87	24	6,25	42,00	69,00
Prioridade	75	12	15,00	28,75	50,33

Política	CPU útil	Sobrecarga	Ocioso	Faltas	Throughput
FCFS	77.14%	13	3	6	0,0429
RR, q=3	62.07%	24	9	5	0,0345
Prioridade	72.00%	12	9	5	0,0400

Interpretação. FCFS encerra o conjunto em 70 unidades, contra 87 do RR. O RR, porém, reduz a resposta média de 17,75 para 6,25: as primeiras oportunidades de CPU chegam antes. O custo é realizar 24 trocas contra 13, além de uma distribuição diferente dos bloqueios e da ociosidade.

Prioridade obtém o menor retorno médio de processos (50,33), mas não o menor tempo total (75). Ela favorece as tarefas com prioridades mais altas; o término de toda a carga ainda depende da tarefa restante. Retorno médio e duração total medem propriedades diferentes e não devem ser confundidos.

Mesmo mantendo os parâmetros de memória constantes, FCFS apresenta seis faltas e as demais políticas cinco. O escalonamento altera a ordem global das referências e a permanência das páginas na memória. Essa interação é resultado do modelo integrado, não uma mudança na configuração experimental.

Neste cenário, RR melhora a resposta inicial e reduz a eficiência total; não se conclui que uma política seja sempre superior. Cargas, custos e prioridades diferentes podem mudar o resultado. Sem envelhecimento, prioridade também pode adiar threads menos prioritárias diante de um fluxo contínuo, embora a carga finita deste experimento termine.

07 / Experimento 2: tamanho do quantum

Mesma carga mista, política RR, quatro molduras. Somente o quantum muda; o cenário q=3 do experimento anterior é incluído como referência intermediária.

Quantum	Total	Trocas	Resposta média	CPU útil	Retorno médio
1	122	54	3,25	44.26%	95,67
3	87	24	6,25	62.07%	69,00
6	76	15	10,00	71.05%	61,67

Reduzir q de 6 para 1 melhora a resposta média de 10,00 para 3,25, mas aumenta o total de 76 para 122 e as trocas de 15 para 54. Com custo de contexto 1, q=1 faz a sobrecarga igualar as 54 unidades úteis. A utilização útil cai de 71,05% para 44,26%.

As três execuções têm cinco faltas, de modo que esse indicador não explica a diferença principal. A maior frequência de trocas pesa diretamente; a sobreposição de bloqueios também muda o tempo ocioso (7, 9 e 14 unidades para q=6, 3 e 1). Um quantum deve equilibrar resposta e custo de despacho, não ser escolhido apenas pelo menor tempo de resposta.

08 / Experimento 3: número de molduras

FCFS, páginas de tamanho 16, uma thread e a sequência 0,1,2,0,1,2,0,1,2. Somente a quantidade de molduras muda; nove referências mais CPU 1 somam dez unidades úteis.

Molduras	Faltas	Taxa	Total	Sobrecarga	Ocioso
2	9	100.00%	65	10	45
3	3	33.33%	29	4	15

Com duas molduras, cada referência precisa de carregamento: a página necessária foi expulsa pelo ciclo anterior. Com três, as três primeiras referências faltam e as seis restantes acertam. O total cai de 65 para 29 unidades, redução de 55,38%.

A diferença de 36 unidades é explicável: seis faltas a menos economizam 30 unidades de atendimento e seis despachos de retomada economizam outras seis. Mais memória reduz tanto o bloqueio quanto a sobrecarga de retomada nesta carga. Esse resultado específico não prova monotonicidade geral de FIFO; a análise se restringe à sequência fornecida.

09 / Validação e reprodução

A versão anterior passou em 191 verificações com JDK 17 e -Xlint:all. O alvo agora é --release 27; a compilação e os testes com JDK 27 ainda não foram executados neste ambiente. Inclui testes de transições, ordenação, quantum, métricas exatas, tradução, FIFO, isolamento, faltas simultâneas, E/S B/NB, permissões, descritores, blocos, escrita sem espaço e validação de entrada. Integração executa sete cargas nas três políticas e compara duas execuções de cada configuração.

A carga manual verifica P1 com CPU 3 e chegada 0, P2 com CPU 2 e chegada 1, sem custo de contexto. FCFS termina P1 em 3 e P2 em 5; RR $q=2$ termina P2 em 4 e P1 em 5. Ambas usam cinco unidades úteis. Os valores de espera e resposta estão no README e são comparados nos testes.

Comandos principais. Em Linux/macOS: `sh scripts/testar.sh`. Em Windows: powershell -ExecutionPolicy Bypass -File scripts/testar.ps1. Para reproduzir as comparações: `python3 scripts/experimentos.py resultados/nova-comparacao` (python ou py -3 no Windows). A pasta de destino deve ser nova.

O ZIP contém os logs completos em resultados/experimentos. O código não sobrescreve uma pasta não vazia. Resultados gerados e arquivos binários estão ignorados pelo Git. O README explica a compilação, as opções, a linguagem da carga e os cálculos.

10 / Limitações e próximos passos

A simulação tem uma CPU, tempos fixos e ausência de instruções condicionais. O paginador é independente do disco genérico. Arquivos ficam em memória, sem persistência física e sem custo de E/S associado. Não há usuários, TLB, páginas sujas, multiprocessamento, aging, cache, journaling ou escalonamento avançado de disco. São simplificações declaradas, não funcionalidades parcialmente implementadas.

A cobertura verifica cenários relevantes, mas não constitui prova formal de correção. Os tempos são unidades abstratas: não podem ser interpretados como desempenho de hardware real. Os resultados se referem às cargas entregues; generalizações exigiriam novos experimentos controlados.

A publicação do repositório GitHub acessível ao professor, os nomes dos integrantes e o histórico real de participação devem ser providenciados pelo grupo. Não foi criado histórico artificial nem simulada uma entrega intermediária. Antes da submissão, o grupo deve executar o projeto em seu próprio ambiente e preparar a explicação de cada decisão.

11 / Fontes e ferramentas

Requisitos: Trabalho SO.pdf, disciplina de Sistemas Operacionais, Prof. Maurício Acconcia Dias, fornecido pelo usuário. Estilo: Markdown.md colado, fornecido pelo usuário. Ferramenta de apoio: ChatGPT/Codex, declarada para revisão acadêmica. Java padrão implementa o simulador; Python padrão automatiza experimentos. ReportLab é usado somente no gerador opcional deste PDF e não é dependência de execução do simulador.

12 / Adendo: desktop gráfico

Foi acrescentada uma interface Swing, usando o módulo padrão `java.desktop`. O visual adota fundo verde-petróleo, títulos azul-marinho, painéis cinza e bordas em relevo, com organização em janelas internas e barra de tarefas. A inspiração combina desktops como KDE Plasma com a estética clássica do Windows 98; nenhum componente desses sistemas foi incorporado.

`InterfaceGrafica` reúne seleção e prévia de cargas, política de CPU, quantum, custos, memória, dispositivos, semente e pasta de saída. Após executar, o monitor apresenta métricas, processos, threads, linha do tempo da CPU e registro completo. Cada execução gera uma pasta própria com os mesmos logs e CSVs da CLI.

O fluxo `Main.executar` é compartilhado por CLI e GUI. A simulação é executada por `SwingWorker` para manter a janela responsiva; o núcleo continua sequencial, orientado a eventos e clock lógico. O botão de execução fica desabilitado durante o trabalho. Não há execução real de processos do sistema hospedeiro.

`LinhaTempo` projeta eventos concluídos: `INSTRUCAO` representa `[clock-1, clock]` e os eventos de troca delimitam a sobrecarga. A visualização não é uma reprodução em tempo real nem depuração passo a passo. O log conserva o detalhe exato; a escala gráfica comprime cargas longas.

O pacote Windows passa a oferecer `SimuladorSO.exe` para a GUI, `SimuladorCLI.exe` para automação e `Testar.exe` para a suíte. `INICIAR.bat` abre a GUI; `MENU-CLI.bat` preserva o menu textual. O runtime embutido inclui `java.base` e `java.desktop`. É preciso regenerar o pacote no Windows para obter esses executáveis.

Validação. A compilação de compatibilidade com JDK 17 passou nas 191 verificações do núcleo. Um teste offscreen acionou a interface real, verificou o bloqueio de execução duplicada e comparou os três CSVs de métricas com o fluxo da CLI. As capturas da interface em docs foram produzidas nesse teste. O alvo entregue permanece Java 27; a abertura e interação nativa no Windows/JDK 27 ainda precisam de conferência.

O diagrama de classes foi atualizado com `InterfaceGrafica` e `LinhaTempo`. Instruções completas e limitações estão em `docs/interface-grafica.md`. O pacote contém fontes e gerador; o executável Windows não foi produzido neste ambiente Linux.